

Live Lecture – Week 4

- As usual
 - Revising last week
 - Q&A for last week
 - Preview for this week
- Reminder: **Assignment is due on Tuesday 10th Oct 12noon**
- Today: More about **Spelling Correction + Soundex**

Wild-card queries: *

- ***mon****: find all docs containing any word beginning with “mon”.
- Easy with binary tree (or B-tree) lexicon: retrieve all words in range: ***mon ≤ w < moo***
- ****mon***: find words ending in “mon”: harder
 - Maintain an additional B-tree for terms *backwards*.
Can retrieve all words in range: ***nom ≤ w < non***.

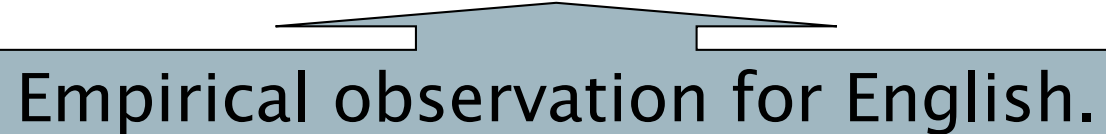
Exercise: from this, how can we enumerate all terms meeting the wild-card query ***pro*cent***?

B-trees handle *'s at the end of a query term

- How can we handle *'s in the middle of query term?
 - *co*tion*
- We could look up *co** AND **tion* in a B-tree and intersect the two term sets
 - Expensive
- The solution: transform wild-card queries so that the *'s occur at the end
- This gives rise to the **Permuterm** Index.

Permuterm query processing

- Rotate query wild-card to the right
- Now use B-tree lookup as before.
- *Permuterm problem: $\approx$ quadruples lexicon size*



Empirical observation for English.

Bigram (k -gram) indexes

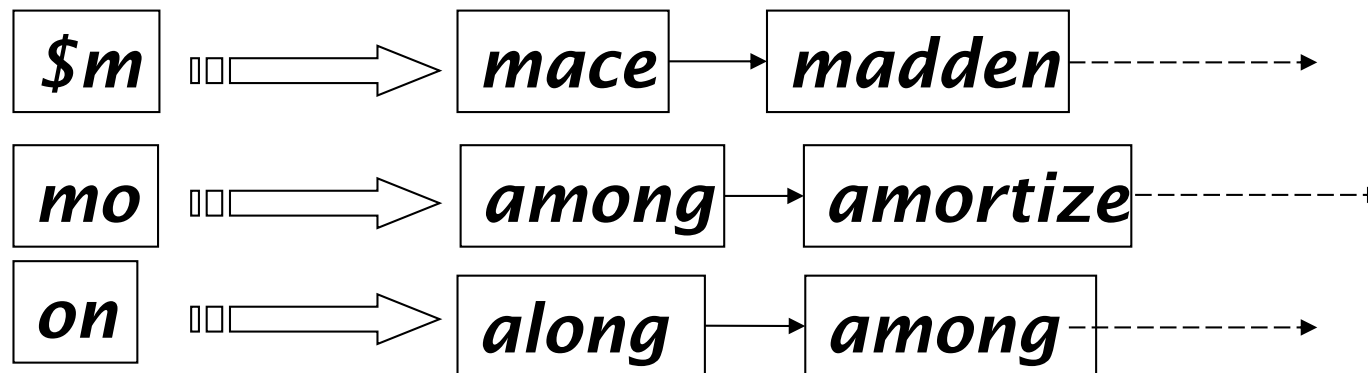
- Enumerate all k -grams (sequence of k chars) occurring in any term
- e.g., from text “***April is the cruelest month***” we get the 2-grams (*bigrams*)

\$a,ap,pr,ri,il,l\$, \$i,is,s\$, \$t,th,he,e\$, \$c,cr,ru,ue,el,le,es,st,t\$, \$m,mo,on,nt,h\$

- \$ is a special word boundary symbol
- Maintain a second inverted index from bigrams to dictionary terms that match each bigram.

Bigram index example

- The k -gram index finds *terms* based on a query consisting of k -grams (here $k=2$).



Processing wild-cards

- Query ***mon**** can now be run as
 - ***\$m AND mo AND on*** 
- Gets terms that match AND version of our wildcard query.
- But we'd enumerate ***moon***.
- Must post-filter these terms against query.
- Surviving enumerated terms are then looked up in the term-document inverted index.
- Fast, space efficient (compared to permuterm).

Processing wild-card queries

- As before, we must execute a Boolean query for each enumerated, filtered term.
- Wild-cards can result in expensive query execution (very large disjunctions...)
 - `pyth*` AND `prog*`
- If you encourage “laziness” people will respond!

Search

Type your search terms, use '*' if you need to.
E.g., `Alex*` will match Alexander.

- Which web search engines allow wildcard queries?

SPELLING CORRECTION (MORE)

Spell correction

- Two principal uses
 - Correcting document(s) being indexed
 - Correcting user queries to retrieve “right” answers
- Two main flavors:
 - Isolated word
 - Check each word on its own for misspelling
 - Will not catch typos resulting in correctly spelled words
 - e.g., *from* → *form*
 - Context-sensitive
 - Look at surrounding words,
 - e.g., *I flew form Heathrow to Narita.*

Document correction

- Especially needed for OCR'ed documents
 - Correction algorithms are tuned for this: rn/m
 - Can use domain-specific knowledge
 - E.g., OCR can confuse O and D more often than it would confuse O and I (adjacent on the QWERTY keyboard, so more likely interchanged in typing).
- But also: web pages and even printed material have typos
- Goal: the dictionary contains fewer misspellings
- But often we don't change the documents and instead fix the query-document mapping

Query mis-spellings

- Our principal focus here
 - E.g., the query ***Alanis Morisset***
- We can either
 - Retrieve documents indexed by the correct spelling, OR
 - Return several suggested alternative queries with the correct spelling
 - *Did you mean ... ?*

Isolated word correction

- Fundamental premise – there is a lexicon from which the correct spellings come
- Two basic choices for this
 - A standard lexicon such as
 - Webster's English Dictionary
 - An “industry-specific” lexicon – hand-maintained
 - The lexicon of the indexed corpus
 - E.g., all words on the web
 - All names, acronyms etc.
 - (Including the mis-spellings)

Isolated word correction

- Given a lexicon and a character sequence Q , return the words in the lexicon closest to Q
- What's "closest"?
- We'll study several alternatives
 - Edit distance (Levenshtein distance)
 - Weighted edit distance
 - n -gram overlap

Edit distance

- Given two strings S_1 and S_2 , the minimum number of operations to convert one to the other
- Operations are typically character-level
 - Insert, Delete, Replace, (Transposition)
- E.g., the edit distance from **dof** to **dog** is 1
 - From **cat** to **act** is 2 (Just 1 with transpose.)
 - from **cat** to **dog** is 3.
- Generally found by dynamic programming.
- See <http://www.merriampark.com/ld.htm> for a nice example plus an applet.

Weighted edit distance

- As above, but the weight of an operation depends on the character(s) involved
 - Meant to capture OCR or keyboard errors
Example: ***m*** more likely to be mis-typed as ***n*** than as ***q***
 - Therefore, replacing ***m*** by ***n*** is a smaller edit distance than by ***q***
 - This may be formulated as a probability model
- Requires weight matrix as input
- Modify dynamic programming to handle weights

Using edit distances

- Given query, first enumerate all character sequences within a preset (weighted) edit distance (e.g., 2)
- Intersect this set with list of “correct” words
- Show terms you found to user as suggestions
- Alternatively,
 - We can look up all possible corrections in our inverted index and return all docs ... slow
 - We can run with a single most likely correction
- The alternatives disempower the user, but save a round of interaction with the user

Edit distance to all dictionary terms?

- Given a (mis-spelled) query – do we compute its edit distance to every dictionary term?
 - Expensive and slow
 - Alternative?
- How do we cut the set of candidate dictionary terms?
- One possibility is to use n -gram overlap for this
- This can also be used by itself for spelling correction.

n -gram overlap

- Enumerate all the n -grams in the query string as well as in the lexicon
- Use the n -gram index (recall wild-card search) to retrieve all lexicon terms matching any of the query n -grams
- Threshold by number of matching n -grams
 - Variants – weight by keyboard layout, etc.

Example with trigrams

- Suppose the text is ***november***
 - Trigrams are *nov, ove, vem, emb, mbe, ber.*
- The query is ***december***
 - Trigrams are *dec, ece, cem, emb, mbe, ber.*
- So 3 trigrams overlap (of 6 in each term)
- How can we turn this into a normalized measure of overlap?

One option – Jaccard coefficient

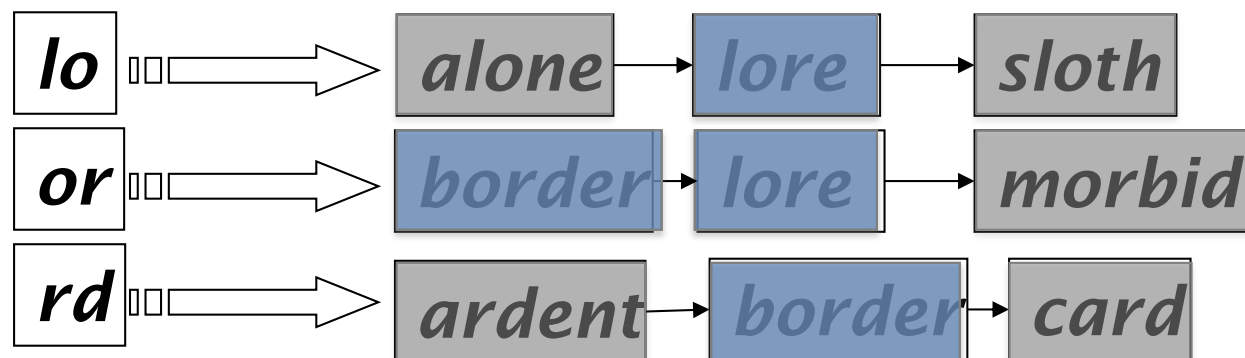
- A commonly-used measure of overlap
- Let X and Y be two sets; then the J.C. is

$$|X \cap Y| / |X \cup Y|$$

- Equals 1 when X and Y have the same elements and zero when they are disjoint
- X and Y don't have to be of the same size
- Always assigns a number between 0 and 1
 - Now threshold to decide if you have a match
 - E.g., if J.C. > 0.8, declare a match

Matching trigrams

- Consider the query **lord** – we wish to identify words matching 2 of its 3 bigrams (**lo**, **or**, **rd**)



Standard postings “merge” will enumerate ...

Adapt this to using Jaccard (or another) measure.

Context-sensitive spell correction

- Text: *I flew from Heathrow to Narita.*
- Consider the phrase query “*flew form Heathrow*”
- We’d like to respond

Did you mean “*flew from Heathrow*”?

because no docs matched the query phrase.

Context-sensitive correction

- Need surrounding context to catch this.
- First idea: retrieve dictionary terms close (in weighted edit distance) to each query term
- Now try all possible resulting phrases with one word “fixed” at a time
 - *flew from heathrow*
 - *fled form heathrow*
 - *flea form heathrow*
- **Hit-based spelling correction:** Suggest the alternative that has lots of hits.

Exercise

- Suppose that for “***flew form Heathrow***” we have 7 alternatives for flew, 19 for form and 3 for heathrow. How many “corrected” phrases will we enumerate in this scheme?

Another approach

- Break phrase query into a conjunction of biwords (Lecture 2).
- Look for biwords that need only one term corrected.
- Enumerate only phrases containing “common” biwords.

General issues in spell correction

- We enumerate multiple alternatives for “Did you mean?”
- Need to figure out which to present to the user
 - The alternative hitting most docs
 - Query log analysis
- More generally, rank alternatives probabilistically

$$\operatorname{argmax}_{corr} P(corr \mid query)$$

- From Bayes rule, this is equivalent to

$$\operatorname{argmax}_{corr} P(query \mid corr) * P(corr)$$

Noisy channel

Language model

SOUNDEX

Soundex

- Class of heuristics to expand a query into **phonetic** equivalents
 - Language specific – mainly for names
 - E.g., ***chebyshev*** → ***tchebycheff***
- Invented for the U.S. census ... in 1918

Soundex – typical algorithm

- Turn every token to be indexed into a 4-character reduced form
- Do the same with query terms
- Build and search an index on the reduced forms
 - (when the query calls for a soundex match)
- <http://www.creativyst.com/Doc/Articles/SoundEx1/SoundEx1.htm#Top>

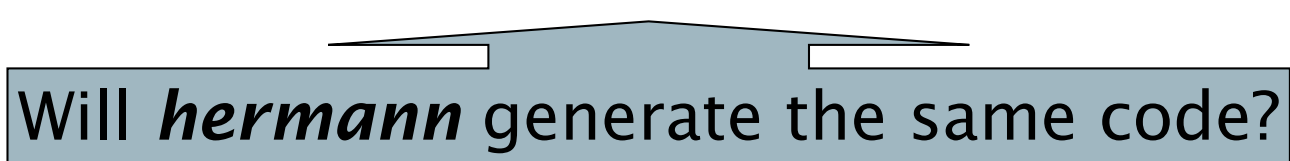
Soundex – typical algorithm

1. Retain the first letter of the word.
2. Change all occurrences of the following letters to '0' (zero):
'A', 'E', 'I', 'O', 'U', 'H', 'W', 'Y'.
3. Change letters to digits as follows:
 - B, F, P, V $\rightarrow$ 1
 - C, G, J, K, Q, S, X, Z $\rightarrow$ 2
 - D, T $\rightarrow$ 3
 - L $\rightarrow$ 4
 - M, N $\rightarrow$ 5
 - R $\rightarrow$ 6

Soundex continued

4. Remove all pairs of consecutive digits.
5. Remove all zeros from the resulting string.
6. Pad the resulting string with trailing zeros and return the first four positions, which will be of the form <uppercase letter> <digit> <digit> <digit>.

E.g., ***Herman*** becomes H655.



Will ***hermann*** generate the same code?

Soundex

- Soundex is the classic algorithm, provided by most databases (Oracle, Microsoft, ...)
- How useful is soundex?
- Not very – for information retrieval
- Okay for “high recall” tasks (e.g., Interpol), though biased to names of certain nationalities
- Zobel and Dart (1996) show that other algorithms for phonetic matching perform much better in the context of IR

Python examples/exercises

- Mapping from the formal knowledge/concepts from the lectures to Python is not so challenging, as Python has heaps of libraries. There are also many useful resources (blogs, tutorials, forums, stackoverflow, ...) on the Internet.
- For example,

Setting up the libraries examples

```
from jellyfish import soundex, metaphone, nysiis, match_rating_codex,\
    levenshtein_distance, damerau_levenshtein_distance, hamming_distance,\
    jaro_similarity
from itertools import groupby
import pandas as pd
import numpy as np

dataList = ['two', 'too', 'to', 'fourth', 'forth', 'dessert',
            'desert', 'Byrne', 'Boern', 'Smith', 'Smyth', 'Catherine', 'Kathryn']

sounds_encoding_methods = [soundex, metaphone, nysiis, match_rating_codex]
```

A soundex example from:

<https://stackoverflow.com/questions/66715423/distance-between-strings-by-similarity-of-sound>

Let compare different phonetic encoding

```
report = pd.DataFrame([dataList]).T
report.columns = ['word']
for i in sounds_encoding_methods:
    print(i.__name__)
    report[i.__name__] = report['word'].apply(lambda x: i(x))
print(report)
```

	soundex	metaphone	nysiis	match_rating_codex
word				
two	T000	TW	TW	TW
too	T000	T	T	T
to	T000	T	T	T
fourth	F630	FR0	FART	FRTH
forth	F630	FR0	FART	FRTH
dessert	D263	TSRT	DASAD	DSRT
desert	D263	TSRT	DASAD	DSRT
Byrne	B650	BRN	BYRN	BYRN
Boern	B650	BRN	BARN	BRN
Smith	S530	SM0	SNAT	SMTH
Smyth	S530	SM0	SNYT	SMYTH
Catherine	C365	K0RN	CATARAN	CTHRN
Kathryn	K365	K0RN	CATRYN	KTHRYN

Soundex (con't)

```
>>> from nltk.stem.porter import *
```

Create a new Porter stemmer.

```
>>> stemmer = PorterStemmer()
```

Test the stemmer on various pluralised words.

```
>>> plurals = ['caresses', 'flies', 'dies', 'mules', 'denied',  
...           'died', 'agreed', 'owned', 'humbled', 'sized',  
...           'meeting', 'stating', 'siezing', 'itemization',  
...           'sensational', 'traditional', 'reference', 'colonizer',  
...           'plotted']
```

```
>>> singles = [stemmer.stem(plural) for plural in plurals]
```

```
>>> print(' '.join(singles))  
caress fli die mule deni die agre own humbl size meet  
state siez item sensat tradit refer colon plot
```

Porter stemmer example from <https://www.nltk.org/howto/stem.html>

```
import nltk
from nltk.stem import WordNetLemmatizer

# Init the Wordnet Lemmatizer
lemmatizer = WordNetLemmatizer()

# Lemmatize Single Word
print(lemmatizer.lemmatize("bats"))
#> bat

print(lemmatizer.lemmatize("are"))
#> are

print(lemmatizer.lemmatize("feet"))
#> foot
```

Lemmatization example from
<https://www.machinelearningplus.com/nlp/lemmatization-examples-python/>

Exercise

- Draw yourself a diagram showing the various indexes in a search engine incorporating all the functionality we have talked about
- Identify some of the key design choices in the index pipeline:
 - Does stemming happen before the Soundex index?
 - What about n -grams?
- Given a query, how would you parse and dispatch sub-queries to the various indexes?

Resources

- IIR 3, MG 4.2
- Efficient spell retrieval:
 - K. Kukich. Techniques for automatically correcting words in text. ACM Computing Surveys 24(4), Dec 1992.
 - J. Zobel and P. Dart. Finding approximate matches in large lexicons. Software - practice and experience 25(3), March 1995.
<http://citeseer.ist.psu.edu/zobel95finding.html>
 - Mikael Tillenius: Efficient Generation and Ranking of Spelling Error Corrections. Master's thesis at Sweden's Royal Institute of Technology.
<http://citeseer.ist.psu.edu/179155.html>
- **Nice, easy reading on spell correction:**
 - Peter Norvig: How to write a spelling corrector
<http://norvig.com/spell-correct.html>

Preview - next lecture: index construction

- Sort-based indexing
 - Naïve in-memory inversion
 - Blocked Sort-Based Indexing (BSBI)
 - Merge sort is effective for hard disk-based sorting (avoid seeks!)
- Single-Pass In-Memory Indexing (SPIMI)
 - No global dictionary
 - Generate separate dictionary for each block
 - Don't sort postings
 - Accumulate postings in postings lists as they occur
- Distributed indexing using MapReduce (very briefly)
- Dynamic indexing: Multiple indices, logarithmic merge